

Git Flow, Conventional Commits y Versionado Semantico

Guia completa para un flujo de trabajo profesional en equipo
Lectura completa: ~60 minutos
Parte 1 — Conventional Commits y Validacion

1. Conventional Commits

Conventional Commits es una especificacion ligera para mensajes de commit que permite:

- Generar **CHANGELOGs automaticamente**
- Determinar cambios de **version semantica automaticamente**
- Mejorar la **comunicacion** entre desarrolladores, DevOps y herramientas automatizadas

1.1 Formato del mensaje

```
<tipo>[<ambito>][!]: <asunto>

[cuerpo]

[BREAKING CHANGE: <descripcion>]

[notas de pie]
```

Regla de commit que determina el tipo de cambio

```
# Para un FIX (patch): commit con tipo fix
fix: corregir calculo de premios en sorteo

# Para un FEATURE (minor): commit con tipo feat
feat: agregar exportacion de resultados a PDF

# Para un BREAKING CHANGE (major): !
refactor!(auth): cambiar de RSA a JWT

# 0 con BREAKING CHANGE en el pie de commit:
feat(api): cambiar estructura de respuesta

BREAKING CHANGE: Campo 'data' ahora es un objeto
```

1.2 Tipos principales

Tipo	Descripcion	Version SemVer
feat	Nueva funcionalidad visible para el usuario	MINOR (1.0.0 → 1.1.0)
fix	Correccion de un bug (error conocido)	PATCH (1.0.0 → 1.0.1)
BREAKING CHANGE	Cualquier cambio que rompa compatibilidad hacia atras	MAJOR (1.0.0 → 2.0.0)

1.3 Tipos adicionales

Tipo	Cuando usar	Ejemplo
build	Sistema de build o dependencias	build: upgrade gradle a 8.0
chore	Trabajo de mantenimiento	chore: limpiar cache de dependencias
ci	Integracion continua	ci: agregar step de codegen en CI
docs	Documentacion	docs: actualizar guia de instalacion
perf	Mejora de rendimiento	perf: cachear consultas de base de datos
refactor	Refactorizacion (sin cambio de funcionalidad)	refactor: extraer widget comun
style	Cambios de formato	style: dart format en todo el proyecto
test	Agregar o corregir tests	test: cubrir caso borde en sorteo

1.4 Reglas de los asuntos (subjects)

- Usar **imperativo presente**: "agregar feature", no "agregue feature" ni "haciendo feature"
- **Sin punto final**
- Maximo **72 caracteres**
- **Primera letra en minusculas**

1.5 El ambito (scope)

Opcional, agrega contexto adicional:

```
feat(raffles): agregar filtro por fecha
fix(auth): corregir login con Google
feat(export): agregar exportacion PDF
```

1.6 Breaking Changes

```
# Opcion 1: ! despues del tipo
refactor!(auth): cambiar respuesta de login a JWT
```

```
# Opcion 2: BREAKING CHANGE en el pie
feat: migrar de SQLite a Isar
```

```
BREAKING CHANGE: Se elimino la base de datos local. Los usuarios deben exportar sus datos antes de actualizar.
```

1.7 Notas al pie

```
feat: agregar autenticacion con Google
```

```
Closes #123, Fixes #456, Refs #789
```

1.8 Lenguaje natural

Se pueden agregar colas de texto despues de los prefijos:

```
feat: agregar autenticacion con Google y Facebook
```

1.9 COMMIT_MESSAGES.md para el proyecto

Crear un archivo COMMIT_MESSAGES.md en la raiz del proyecto para estandarizar:

```
# Conventional Commits para [Nombre del Proyecto]

## Tipos permitidos
- feat: Nueva funcionalidad
- fix: Correccion de bug
- refactor: Refactorizacion sin cambio funcional
- docs: Documentacion
- test: Tests
- chore: Mantenimiento
- build: Build system
- ci: CI/CD
- perf: Rendimiento
- style: Formato

## Ambitos comunes
- auth: Autenticacion
- raffles: Sorteos
- export: Exportacion
- ui: Interfaz de usuario

## Ejemplo
feat(raffles): agregar filtro por fecha en lista de sorteos
```

1.10 Relacion con SemVer

Conventional Commits permite calcular automaticamente la version SemVer:

```
# Version actual: 1.0.0

# Si el ultimo commit es:
fix: corregir calculo → 1.0.1 (PATCH)
feat: agregar filtro → 1.1.0 (MINOR)
feat!: cambiar API → 2.0.0 (MAJOR)

# Con standard-version, esto es automatico
standard-version --first-release
```

1.11 Especificacion Completa (Reglas 1-16)

Las palabras clave "DEBE", "NO DEBE", "REQUERIDO", "DEBERÁ", "NO DEBERÁ", "DEBERÍA", "NO DEBERÍA", "RECOMIENDA", "PUEDE" y "OPCIONAL" se interpretan como se describe en RFC 2119.

#	Regla
---	-------

1	Los commits DEBEN iniciarse con un prefijo de tipo que consiste en un sustantivo, feat, fix, etc., seguido del ámbito OPCIONAL, ! OPCIONAL, y dos puntos y un espacio REQUERIDO.
2	El tipo feat DEBE ser usado cuando un commit agrega una nueva funcionalidad a la aplicación o librería.
3	El tipo fix DEBE ser usado cuando el commit representa una corrección a un error en el código de la aplicación (bug).
4	Un ámbito PUEDE ser añadido después de un tipo. Un ámbito DEBE consistir en un sustantivo que describa una sección de la base del código encerrado entre paréntesis, ej., fix(parser):.
5	Una descripción DEBE ir inmediatamente después de los dos puntos y el espacio del prefijo de tipo/ámbito. La descripción es un resumen corto de los cambios realizados en el código.
6	Un cuerpo de commit más extenso PUEDE agregarse después de la descripción corta, dando información contextual adicional acerca de los cambios en el código. El cuerpo DEBE iniciar después de una línea en blanco después de la descripción.
7	Un cuerpo de commit es de forma-libre y PUEDE consistir de cualquier número de párrafos separados por una nueva línea.
8	Una o más notas al pie PUEDEN ser añadidas una línea en blanco después del cuerpo. Cada nota al pie DEBE consistir de una palabra clave, seguida ya sea por un separador :<espacio> o <espacio>#, seguido por un valor cadena (string).
9	Una palabra clave de una nota al pie DEBE usar - en lugar de caracteres de espacios en blanco, ej., Acked-by. Se hace una excepción para BREAKING CHANGE.
10	Una nota al pie PUEDE contener espacios y líneas en blanco, y el parseo DEBE terminar cuando se observe el siguiente separador/clave.
11	Los cambios de ruptura DEBEN ser indicados en el prefijo de tipo/ámbito de un commit, o como una entrada en la nota al pie.
12	Si se incluye como una nota al pie, un cambio de ruptura DEBE consistir del texto en mayúsculas BREAKING CHANGE, seguido de dos puntos, y una descripción.
13	Si se incluye en el prefijo de tipo/ámbito, cambios de ruptura DEBEN ser indicados por un ! inmediatamente después de :. Si ! es usado, BREAKING CHANGE: PUEDE ser omitido de la sección de la nota al pie.
14	Tipos diferentes a feat y fix PUEDEN ser usados en los mensajes de commit, ej., docs: updated ref docs.
15	Las unidades de información que componen Commits Convencionales NO DEBEN ser tratados como implementadores sensitivos de caso, con la excepción de BREAKING CHANGE que DEBE ir en mayúsculas.
16	BREAKING-CHANGE DEBE ser sinónimo de BREAKING CHANGE, cuando se usa en una nota al pie.

1.12 Ejemplos Adicionales de la Especificacion

```
# Mensaje con cambio de ruptura en la nota al pie
feat: allow provided config object to extend other configs
BREAKING CHANGE: `extends` key in config file is now used for extending other config files

# Mensaje con ! para llamar atencion al cambio de ruptura
refactor!: drop support for Node 6

# Mensaje con ambos ! y BREAKING CHANGE en la nota al pie
refactor!: drop support for Node 6
BREAKING CHANGE: refactor to use JavaScript features not available in Node 6.

# Mensaje sin cuerpo
docs: correct spelling of CHANGELOG

# Mensaje con ámbito
feat(lang): added polish language

# Mensaje con cuerpo multi-párrafo y múltiples notas al pie
fix: correct minor typos in code
see the issue for details
on typos fixed.
Reviewed-by: Z
Refs #133
```

1.13 Preguntas Frecuentes (FAQ)

¿Cómo puedo trabajar con los mensajes de los commits en la etapa inicial de desarrollo?

Recomendamos trabajar como si ya hubieras lanzado tu producto. Típicamente alguien, incluso si son tus compañeros desarrolladores, están usando tu software. Ellos querrán saber qué se ha arreglado, qué se ha dañado, etc.

¿Los tipos en los títulos de los commits están en mayúsculas o en minúsculas?

Se puede utilizar cualquiera, pero es mejor ser coherente.

¿Qué hago si el commit se ajusta a más de uno de los tipos de commit?

Regresa y haz múltiples commits de ser posible. Parte de los beneficios de los Commits Convencionales es la habilidad para hacer commits y PRs más organizados.

¿No desalienta esto el desarrollo y la iteración rápida?

Desalienta moverse rápido de una forma desorganizada. Te ayuda a ser capaz de moverte rápido a largo plazo a través de proyectos con una gran variedad de contribuidores.

¿Pueden los Commits Convencionales llevar a los desarrolladores a limitar el tipo de commits que hacen?

Los Commits Convencionales nos animan a hacer más de ciertos tipos de commits como fixes. Adicionalmente, la flexibilidad de los Commits Convencionales le permite a tu equipo generar sus propios tipos y cambiarlos a lo largo del tiempo.

¿Cómo se relaciona esto con SemVer?

El tipo de commit fix se traduce a un cambio de versión PATCH. El tipo de commit feat se traduce a un cambio de versión MINOR. Commits con el texto BREAKING CHANGE, sin importar su tipo, se traducen a un cambio de versión MAJOR.

¿Qué debo hacer si por accidente uso un tipo de commit equivocado?

Cuando utilizaste un tipo que es de la especificación pero del tipo correcto, ej. fix en lugar de feat: Antes de combinar o liberar el error, recomendamos usar git rebase -i para editar el historial de los commits. Después de que se ha liberado, la limpieza será distinta de acuerdo a las herramientas y procesos que hayas utilizado.

Cuando usas un tipo que no está en la especificación, ej. feet en lugar de feat: En el peor de los escenarios, no es el fin del mundo si aparece un commit que no cumple con las especificaciones. Simplemente significa que el commit será ignorado por las herramientas que se basen en esta especificación.

¿Deben todos los que contribuyen a mi proyecto usar esta especificación?

¡No! Si usas un flujo de trabajo basado en squash los líderes del proyecto pueden limpiar el mensaje en el momento en que se incorpora, sin agregar cargas adicionales a quienes contribuyen casualmente.

¿Cómo maneja Commits Convencionales el revertir código?

Commits Convencionales no hace un esfuerzo explícito para definir el comportamiento al revertir. En lugar de eso le dejamos a los autores de las herramientas la flexibilidad de los tipos y notas al pie para desarrollar la lógica para manejar los reverts. Una recomendación es usar el tipo revert, y una nota al pie que haga referencia a los commit SHAs que son revertidos:

```
revert: let us never again speak of the noodle incident  
Refs: 676104e, a215868
```

Licencia: Creative Commons - CC BY 3.0

2. Husky + lint-staged

Husky ejecuta scripts automaticos en hooks de Git. **lint-staged** ejecuta comandos SOLO en archivos staged (preparados para commit).

2.1 Instalacion

```
# Husky (moderno, v9+)  
npm install --save-dev husky  
npx husky init  
# Crea .husky/ con hooks de ejemplo  
  
# lint-staged  
npm install --save-dev lint-staged
```

2.2 Configuracion en package.json

```
{
  "scripts": {
    "prepare": "husky"
  },
  "lint-staged": {
    "*.dart": [
      "dart format --set-exit-if-changed",
      "flutter analyze --fatal-infos"
    ],
    "*.{yaml,json}": [
      "prettier --write"
    ]
  }
}
```

2.3 Los 3 hooks principales

Hook	Cuando se ejecuta	Que hace	Importancia
pre-commit	Antes del commit	Ejecuta lint-staged (analisis + formato)	CALIDAD
commit-msg	Despues del mensaje	Valida formato (commitlint)	CONVENCION
pre-push	Antes del push	Ejecuta tests completos	FIABILIDAD

2.4 Configuracion del pre-commit

```
#!/usr/bin/env sh
. "$(dirname -- "$0")/_/husky.sh"

npx lint-staged
```

2.5 Configuracion del commit-msg

```
#!/usr/bin/env sh
. "$(dirname -- "$0")/_/husky.sh"

npx --no -- commitlint --edit ${1}
```

2.6 Configuracion del pre-push

```
#!/usr/bin/env sh
. "$(dirname -- "$0")/_/husky.sh"

flutter test
```

2.7 Orden de ejecucion

```
$ git commit -m "feat: agregar filtro"

# 1. Husky ejecuta pre-commit
→ dart format --set-exit-if-changed
→ flutter analyze --fatal-infos

# 2. Husky ejecuta commit-msg
→ commitlint --edit .git/COMMIT_EDITMSG

# 3. Commit exitoso ✓
```

2.8 Configuración avanzada de lint-staged

```
{
  "lint-staged": {
    "*.dart": {
      "dart format --set-exit-if-changed": true,
      "flutter analyze --fatal-infos": true
    },
    "*.yaml": ["yamllint"],
    "*.json": ["prettier --write"],
    "*.md": ["prettier --write"]
  }
}
```

2.9 Depuración de Husky

```
# Verificar que Husky está instalado
ls .husky/

# Verificar los hooks
cat .husky/pre-commit
cat .husky/commit-msg

# Ejecutar manualmente un hook
npx lint-staged
```

2.10 Emergencias: saltarse los hooks

```
# Saltar todos los hooks (no recomendado)
git commit --no-verify -m "fix: hotfix critico"

# Forzar commit sin validación
git commit -am "fix: fix urgente" --no-verify
```

2.11 Consideraciones para CI/CD


```
# En GitHub Actions, Husky NO se ejecuta automaticamente
# Los hooks solo corren en el entorno local

# Solucion: Ejecutar las mismas validaciones en CI
- name: Validate commits
  run: npx commitlint --from ${ github.event.pull_request.base.sha } --to ${ github.event.pull_request.head.sha }

- name: Lint staged
  run: npx lint-staged
```

2.12 Reconocimiento de committer con .mailmap

```
# Archivo .mailmap en la raiz del repo
# Une multiples identidades de email/usuario

Nombre Real <email_real@example.com> <email_anterion@example.com>
```

3. Commitizen + commitlint

Commitizen = CLI interactivo que genera commits con formato correcto. **commitlint** = linter que valida mensajes de commit contra las reglas.

3.1 Instalacion

```
# Commitizen
npm install --save-dev commitizen cz-conventional-changelog

# commitlint
npm install --save-dev @commitlint/cli @commitlint/config-conventional
```

3.2 Configuracion en package.json

```
{
  "scripts": {
    "commit": "cz",
    "prepare": "husky"
  },
  "config": {
    "commitizen": {
      "path": "cz-conventional-changelog"
    }
  }
}
```

3.3 Flujo con Commitizen

```
$ npm run commit
# o
$ npx cz

? Select the type of change:
  feat
? Scope (optional):
  raffles
? Short description:
  agregar filtro por fecha
? Breaking changes (optional):
  No

# Resultado:
feat(raffles): agregar filtro por fecha
```

3.4 Configuración de commitlint

```
// commitlint.config.js
module.exports = {
  extends: ['@commitlint/config-conventional'],
  rules: {
    'type-enum': [2, 'always', [
      'feat', 'fix', 'docs', 'style', 'refactor',
      'perf', 'test', 'build', 'ci', 'chore', 'revert'
    ]],
    'type-empty': [2, 'never'],
    'subject-empty': [2, 'never'],
    'subject-full-stop': [2, 'never', '.'],
    'header-max-length': [2, 'always', 72],
    'body-leading-blank': [2, 'always'],
    'footer-leading-blank': [2, 'always']
  }
};
```

3.5 Reglas de commitlint

Regla	Valor	Descripción
type-enum	[2, 'always', [...]]	Tipos permitidos en commits
type-empty	[2, 'never']	El tipo es obligatorio
subject-empty	[2, 'never']	La descripción es obligatoria
header-max-length	[2, 'always', 72]	Máximo 72 caracteres en la primera línea
subject-full-stop	[2, 'never', '.']	No usar punto final en la descripción

3.6 Estandarizar con COMMIT_MESSAGES.md

```
feat(raffles): agregar filtro por fecha
fix(auth): corregir login con Google
feat(export): agregar exportacion PDF
refactor(ui): extraer widget de sorteo
chore(deps): actualizar dependencias
test(raffles): cubrir caso borde en sorteo
docs(api): documentar endpoint de login
```

3.7 Standard Version: CHANGELOG automatico

```
# Instalar
npm install --save-dev standard-version

# Configurar en package.json
{
  "standard-version": {
    "header": "# CHANGELOG",
    "types": [
      {"type": "feat", "section": "Features"},
      {"type": "fix", "section": "Bug Fixes"},
      {"type": "perf", "section": "Performance"},
      {"type": "refactor", "section": "Refactor"},
      {"type": "docs", "section": "Documentation"}
    ]
  }
}

# Usar
npm run release

# Esto genera:
# - CHANGELOG.md actualizado
# - package.json con version bump
# - Commit con los cambios
# - Tag v1.2.0
```

3.8 Generacion de CHANGELOG con git-cliff

```
# Instalar
cargo install git-cliff

# Ejecutar
cliff -o CHANGELOG.md

# Resultado automatico desde los commits
# basado en Conventional Commits
```

3.9 Integracion completa (Husky + Commitizen + commitlint)

```
# Flujo completo para el desarrollador:
1. npx cz # Genera commit correcto
2. Husky ejecuta pre-commit # Valida formato
3. Husky ejecuta commit-msg # Valida con commitlint
4. git push # Push a origin
5. Pre-push ejecuta tests

# En CI/CD:
1. CI ejecuta commitlint # Valida todos los commits del PR
2. CI ejecuta lint-staged # Valida archivos modificados
3. CI ejecuta flutter test # Valida que no se rompa nada
```

Parte 2 — Versiones, Ramas y Releases

4. FVM: Flutter Version Management

FVM permite gestionar multiples versiones de Flutter en el mismo equipo. Cada proyecto tiene su propia version.

4.1 Instalacion

```
# Instalar FVM
dart pub global activate fvm

# Instalar Flutter 3.24.0
fvm install 3.24.0

# Usar en el proyecto
cd mi_proyecto
fvm use 3.24.0
```

4.2 Comandos esenciales

Comando	Que hace
fvm install <version>	Instala una version de Flutter
fvm use <version>	Configura la version del proyecto actual
fvm flutter pub get	Ejecuta pub get con la version configurada
fvm flutter run	Ejecuta el proyecto con la version configurada
fvm flutter test	Ejecuta tests con la version configurada
fvm list	Lista todas las versiones instaladas
fvm current	Muestra la version actual del proyecto
fvm global <version>	Establece version global del sistema

4.3 Archivos importantes

```
# Archivo de version (SE COMITA)
.fvm/flutter_sdk_version

# SDK instalado (SE IGNORA en .gitignore)
.fvm/flutter_sdk
```

4.4 .gitignore

```
# Agregar a .gitignore
.fvm/flutter_sdk
.fvm/fvm_config.json
```

4.5 Configuración VS Code

```
// .vscode/settings.json
{
  "dart.flutterSdkPath": ".fvm/flutter_sdk",
  "dart.flutterBinaryPath": ".fvm/flutter_sdk/bin/flutter"
}
```

4.6 CI/CD con FVM

```
# GitHub Actions
- name: Setup Flutter
  uses: subosito/flutter-action@v2
  with:
    flutter-version: '3.24.0'

# 0 con FVM
- name: Setup FVM
  run: |
    dart pub global activate fvm
    fvm install
    fvm flutter pub get

- name: Run tests
  run: fvm flutter test
```

4.7 Docker con FVM

```
# Dockerfile
FROM ghcr.io/cirruslabs/flutter:3.24.0

WORKDIR /app
COPY . .
RUN flutter pub get
RUN flutter build apk
```

4.8 Comparativa de herramientas

Herramienta	Ventajas	Desventajas
-------------	----------	-------------

FVM	Rapido, simple, IDE integration	Requiere configuracion inicial
asdf	Multi-lenguaje, plugin system	Mas complejo, overhead
Flutter solo	Sin dependencias extra	Solo una version global

5. Estrategias de Ramas y Releases

Define como organizar el trabajo en equipo: que ramas existen, como se integran, y como se hacen los releases.

5.1 Conventional Branch

Conventional Branch es una especificacion para agregar significado legible a las ramas Git.

```
# Formato general
<tipo>/<descripcion>
```

Prefijos estandar

Prefijo	Proposito	Ejemplo
main / master	Rama principal de desarrollo	main
feature/ o feat/	Nuevas funcionalidades	feature/agregar-login
bugfix/ o fix/	Corregir errores	bugfix/corregir-crash
hotfix/	Correcciones urgentes	hotfix/parche-seguridad
release/	Preparar una liberacion	release/v1.2.0
chore/	Tareas sin codigo	chore/actualizar-deps

Puntos Clave de la Especificacion

- 1. **Nombres basados en un propósito:** El nombre de cada rama indica claramente su propósito, lo que facilita que todos los desarrolladores comprendan para qué sirve la rama.
- 2. **Integración con CI/CD:** Al utilizar nombres de ramas consistentes, puede ayudar a los sistemas automatizados (como canalizaciones de integración continua/implementación continua) a activar acciones específicas basadas en el tipo de rama (por ejemplo, implementación automática desde ramas de liberación).
- 3. **Colaboración en equipo:** Fomenta la colaboración dentro de los equipos al hacer explícito el propósito de la rama, reducir los malentendidos y facilitar que los miembros del equipo cambien entre tareas sin confusión.

Reglas de nomenclatura

- 1. **Alfanuméricos, guiones y puntos en minúsculas:** Utilice siempre letras minúsculas (az), numere (0-9) y guiones (-) para separar palabras. Evite caracteres, guiones bajos o espacios especiales. Para ramas de liberación, puntos (.) pueden usarse en la descripción para representar números de versión (ej., release/v1.2.0).
- 2. **Sin guiones o puntos consecutivos, principales o finales:** Asegúrese de que los guiones y los puntos no aparezcan consecutivamente (ej., feature/nuevo--inicio-de-sesión, release/v1.-2.0), ni al inicio ni al final de la

descripción (ej., feature/-nuevo-inicio-de-sesión, release/v1.2.0.).

3. **Mantenlo claro y conciso:** El nombre de la rama debe ser descriptivo pero conciso, indicando claramente el propósito del trabajo.
4. **Incluir números de ticket:** Si corresponde, incluya el número de ticket de su herramienta de gestión de proyectos para facilitar el seguimiento. Por ejemplo, para un ticket número 123, el nombre de la rama podría ser feature/123-nuevo-login.

Beneficios de Conventional Branch

- **Comunicación clara:** El nombre de la rama por sí solo proporciona una comprensión clara de su propósito en el cambio de código.
- **Fácil de automatizar:** Se conecta fácilmente a procesos de automatización (por ejemplo, diferentes flujos de trabajo para feature, release, etc.).
- **Escalabilidad:** Funciona bien en equipos grandes donde muchos desarrolladores trabajan en diferentes tareas simultáneamente.

Preguntas Frecuentes

¿Por qué los tipos de ramas no son tan detallados como los commits convencionales (ej., build, ci, docs, style, refactor)?

Las ramas son diferentes de las confirmaciones, son temporales y se usan principalmente hasta que se fusionan. Introducir demasiados tipos de ramas sería innecesario y las haría más difíciles de gestionar y recordar.

¿Qué herramientas se pueden utilizar para identificar automáticamente si un miembro del equipo no cumple con esta especificación?

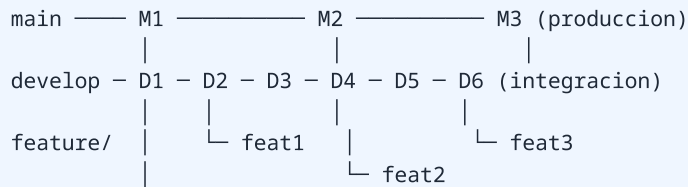
Puedes usar verificación de confirmación para consultar las especificaciones de la rama o confirmar-verificar-acción si sus códigos están alojados en GitHub.

Licencia: Creative Commons - CC BY 3.0

5.2 Git Flow vs GitHub Flow

Aspecto	Git Flow	GitHub Flow
Ramas	main, develop, feature/*, release/*, hotfix/*	main, feature/*
Complejidad	Alta	Baja
Releases	Programados	Continuos
Equipo	5+ personas	1-3 personas
Proyecto	App con versiones semanticas	Web/API con deploys continuos

5.3 Git Flow simplificado para Flutter



5.4 Flujo de trabajo completo

```

# 1. Nueva Feature
git checkout develop
git pull
git checkout -b feature/agregar-filtro-rifas
# ... trabajo ...
git add .
git commit -m "feat(raffles): agregar filtro por fecha"
git push origin feature/agregar-filtro-rifas

# 2. PR a develop (code review)

# 3. Release
git checkout develop
git checkout -b release/1.2.0
# ... ajustes finales ...
git checkout main
git merge release/1.2.0
git tag -a v1.2.0 -m "Release v1.2.0"
git push origin main --tags

# 4. Hotfix
git checkout main
git checkout -b hotfix/corregir-crash
# ... fix ...
git merge hotfix/corregir-crash
git tag -a v1.2.1 -m "Hotfix v1.2.1"
git push origin main --tags

```

5.5 Pull Requests y Code Review

```

## Descripcion

## Cambios
- [x] Nueva feature: filtro por fecha
- [x] Tests unitarios agregados

## Testing
- [x] Unit tests
- [x] Widget tests
- [x] Prueba manual en Android

## Checklist
- [x] Codigo sigue el estilo del proyecto
- [x] No hay warnings de analisis
- [x] Tests pasan

```

5.6 Versionado Semantico (SemVer 2.0.0)

SemVer es un sistema de versionado que comunica cambios significativos a través de números de versión.

Introduccion: El Infierno de Dependencias

En el mundo de la administración de software existe un temido lugar llamado "Infierno de Dependencias". Mientras más crece tu sistema y más paquetes integras dentro de tu software, más probable se hace que un día te encuentres en este pozo de desesperación.

En sistemas con muchas dependencias, lanzar nuevas versiones de los paquetes puede convertirse en una pesadilla. Si las especificaciones de las dependencias son muy estrictas, estarás en peligro de bloquear una versión (la inhabilidad de actualizar un paquete sin tener que publicar una nueva versión de cada otro paquete dependiente). Si las dependencias son especificadas de forma muy relajada, inevitablemente serás mordido por versiones promiscuas (asumir la compatibilidad con próximas versiones más allá de lo razonable).

El Infierno de Dependencias es donde estás cuando una versión bloqueada y/o promiscua previenen que muevas tu proyecto adelante de forma fácil y segura.

Formato de Version

```
v<major>.<minor>.<patch>

v1.0.0  → Primer release estable
v1.1.0  → Nueva feature
v1.1.1  → Bug fix
v2.0.0  → Breaking change
```

Reglas de incremento

Version	Cuando incrementar	Ejemplo
Major (X)	Cambios incompatibles con API anterior	1.0.0 → 2.0.0
Minor (Y)	Nueva funcionalidad compatible	1.0.0 → 1.1.0
Patch (Z)	Correccion de bug compatible	1.1.0 → 1.1.1

Especificacion Completa SemVer 2.0.0

Las palabras clave "DEBE", "NO DEBE", "OBLIGATORIO", "DEBERÁ", "NO DEBERÁ", "DEBERÍA", "NO DEBERÍA", "RECOMENDADO", "PUEDE" y "OPCIONAL" se interpretan como se describe en RFC 2119.

#	Regla
1	El Software que usa Versionado Semántico DEBE declarar un API público. Este API puede ser declarado en el propio código o debe existir estrictamente en la documentación.
2	Un número de versión normal DEBE tener la forma de X.Y.Z donde X, Y y Z son números enteros no negativos, y NO DEBEN ser precedidos de ceros. X es la versión mayor, Y es la versión menor, y Z es la versión parche.
3	Una vez que el paquete versionado ha sido publicado, el contenido de esa versión NO DEBE ser modificado. Cualquier modificación DEBE ser publicada como una nueva versión.
4	Una versión mayor en cero (0.y.z) se considera como desarrollo inicial. Todo PUEDE cambiar en cualquier momento. El API público NO DEBERÍA ser considerado estable.

5	La versión 1.0.0 define el API público. La manera en que cada número de versión es incrementado después de esta publicación dependerá de su API público y cómo cambia.
6	La versión parche Z (x.y.z x > 0) DEBE ser incrementada si solamente se introducen correcciones de errores compatibles con versiones anteriores.
7	La versión menor Y (x.Y.z x > 0) DEBE ser incrementada si se introduce funcionalidad nueva y compatible con la versión anterior del API público. La versión parche DEBE reiniciarse a 0 cuando una versión menor se incrementa.
8	La versión mayor X (X.y.z x > 0) DEBE ser incrementada solamente si se introducen cambios incompatibles con la versión anterior del API público. Versiones parche y menores DEBEN ser reiniciadas a 0 cuando una versión mayor es incrementada.
9	Una versión de prelanzamiento PUEDE ser denotada agregando un guión y una serie de identificadores separados por puntos, inmediatamente seguida de la versión parche. Los identificadores DEBEN ser compuestos sólo de caracteres alfanuméricos ASCII y guión ([0-9A-Za-z-]).
10	Metadatos de compilación PUEDEN ser denotados agregando el signo más y una serie de identificadores separados por puntos, inmediatamente seguido de la versión parche o prelanzamiento. Los metadatos de compilación DEBEN ser ignorados cuando se determina la precedencia de la versión.
11	La precedencia se refiere a cómo las versiones se comparan entre ellas cuando se ordenan. La precedencia DEBE ser calculada separando los identificadores de la versión en mayor, menor, parche y prelanzamiento (dejando de lado los metadatos de compilación).

Precedencia de Versiones

La precedencia es determinada por la primera diferencia al comparar cada uno de los identificadores de izquierda a derecha:

- 1. Mayor, menor y parche son siempre comparadas numéricamente. Por ejemplo: 1.0.0 < 2.0.0 < 2.1.0 < 2.1.1.
- 2. Cuando la versión mayor, menor y parche son iguales, la versión de prelanzamiento tiene menor precedencia que la versión normal. Por ejemplo: 1.0.0-alpha < 1.0.0.
- 3. Identificadores compuestos solamente por números son comparados numéricamente.
- 4. Los identificadores con letras o guiones son comparados léxicamente en orden ASCII.
- 5. Identificadores numéricos siempre tienen menor precedencia que identificadores no numéricos.
- 6. Un conjunto de campos de prelanzamiento más numeroso tiene mayor precedencia que un conjunto menos numeroso.

```
# Ejemplo de precedencia
1.0.0-alpha < 1.0.0-alpha.1 < 1.0.0-alpha.beta < 1.0.0-beta < 1.0.0-beta.2 < 1.0.0-beta.11 < 1.0.0-rc.1 < 1.0.0
```

Versiones de pre-lanzamiento

```
v1.0.0-alpha
v1.0.0-alpha.1
v1.0.0-beta.1
v1.0.0-rc.1
```

Metadatos de compilacion

```
# Formato: v<major>.<minor>.<patch>+<metadata>
v1.0.0-alpha+001
v1.0.0+20130313144700
v1.0.0-beta+exp.sha.5114f85

# Los metadatos son ignorados al determinar precedencia
# v1.0.0+build.1 == v1.0.0+build.2 (misma precedencia)
```

Licencia: Creative Commons - CC BY 3.0

5.7 CHANGELOG

```
# Changelog

## [1.2.0] - 2026-06-15

### Added
- Exportacion de resultados a PDF
- Notificaciones push por sorteo

### Fixed
- Crash al abrir sorteo sin conexion
- Calculo incorrecto de premios

### Changed
- Migracion de SQLite a Isar
```

5.8 Buenas practicas

- **Commits atomicos:** Cada commit es una unidad logica independiente
- **Ramas cortas:** Maximo 2-3 dias de trabajo, PRs < 400 lineas
- **Rebase en feature:** Historia lineal en ramas de feature
- **Merge en compartidas:** Preservar contexto en ramas compartidas
- **main estable:** Solo mergea desde release/ o hotfix/
- **Tags siempre:** Taggear cada merge a main